

Maintain and reuse

Configuration templates, i.e., pre-made modules for infrastructure components are written, reviewed, managed, and reused after storing them under VCS. Organisations can use public templates contributed by the community as well as have unique private templates stored in a central registry.

Init

The Terraform binary contains the basic functionality and everything else is downloaded as and when required. The 'terraform init' step analyses the code, figures out the provider and downloads all the plugins (code) needed by the provider (here, it's AWS). Provider plugins are responsible for interacting over APIs provided by the cloud platforms using the corresponding CLI tools. They are responsible for the life cycle of the resource, i.e., create, read, update and delete. Figure 3 shows the checking and downloading of the provider 'aws' plugins after scanning the configuration file.

Plan

The 'terraform plan' is a dry run for our changes. It builds the topology of all the resources and services needed and, in parallel, handles the creation of dependent and non-dependent resources. It efficiently analyses the previously running state and resources, using a resource graph to calculate the required modifications. It provides the flexibility of validating and scanning infrastructure resources before provisioning, which otherwise would have been risky. The '+' symbol signifies the new resources that will be added to the already existing ones, if any. Figure 4 shows the generation of the plan and shows the changes in resources, with '+' indicating addition and '-' indicating deletion.

Validation

Administrators can validate and approve significant changes produced in 'terraform planning's' dry run. This

```
alakazam@vaibhavs-MacBook-Air auffr $ touch ec2.tf
alakazam@vaibhavs-MacBook-Air auffr $ terraform init

Initializing provider plugins...
- Checking for available provider plugins...
- Downloading plugin for provider "aws" (hashicorp/aws) 2.52.0...

The following providers do not have any version constraints in configuration,
so the latest version was installed.

To prevent automatic upgrades to new major versions that may contain breaking
changes, it is recommended to add version = "..." constraints to the
corresponding provider blocks in configuration, with the constraint strings
suggested below.

* provider.aws: version = "~> 2.52"

Warning: Skipping backend initialization pending configuration upgrade
```

Figure 3: The 'Terraform init' step initialises all the required resources and plugins

```
alakazam@vaibhavs-MacBook-Air auffr $ terraform plan
Refreshing Terraform state in-memory prior to plan...
The refreshed state will be used to calculate this plan, but will not
persisted to local or remote state storage.

-----

An execution plan has been generated and is shown below.
Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.example will be created
+ resource "aws_instance" "example" {
  + ami                  = "ami-13be557e"
  + arn                  = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone     = (known after apply)
  + cpu_core_count       = (known after apply)
  + cpu_threads_per_core  = (known after apply)
  + get_password_data     = false
  + host_id              = (known after apply)
  + id                   = (known after apply)
  + instance_state       = (known after apply)
  + instance_type        = "t2.micro"
  + ipv6_address_count   = (known after apply)
```

Figure 4: 'Terraform plan' dry runs the instantiation

```
alakazam@vaibhavs-MacBook-Air auffr $ terraform apply

An execution plan has been generated and is shown below.
Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.example will be created
+ resource "aws_instance" "example" {
  + ami                  = "ami-13be557e"
  + arn                  = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone     = (known after apply)
  + cpu_core_count       = (known after apply)
```

Figure 5: 'Terraform apply' instantiates the validated infrastructure in the planning step